```
0000 WITH GRMR_OPS, GRMR_TBL;¶USE  GRMR_OPS, GRMR_TBL;¶SEPARATE(¬IDL )¶--|----------------------------------------------------------------------------------------------------------------¶--|¬PROCEDURE¬CHECK_GRMR¶--|----------------------------------------------------------------------------------------------------------------¶PROCEDURE¬CHECK_GRMR ( NOM_TEXTE :STRING ) IS¶  USER_ROOT¬¬: TREE;¶  GR_STATE_SEQ¬¬: SEQ_TYPE;¶¶  STATE¬¬¬: TREE;¶  STATE_NBR¬¬: INTEGER;¶  TER_GO_COUNT¬¬: INTEGER;¶  NONTER_GO_COUNT¬¬: INTEGER;¶  REDUCE_COUNT¬¬: INTEGER;¶¶  REDUCE_NBR_TERS¬¬: ARRAY (1 .. 6) OF¬INTEGER;¶  REDUCE_ITEM¬¬: ARRAY (1 .. 6) OF¬TREE;¶¶  TYPE SYLTBL_TYPE¬¬IS RECORD¶¬¬¬  STATE_NBR¬: INTEGER;¶¬¬¬  REDUCE¬¬: BOOLEAN;¶¬¬¬END RECORD;¶¶  SYLTBL¬¬¬: ARRAY (- INTEGER(170) .. 350) OF SYLTBL_TYPE;¶¶  ALT_SEM_TBL¬¬: ARRAY (0 .. 700) OF INTEGER;¶          -- SEMANTICS FOR ALT (OR 0)¶¶  --|----------------------------------------------------------------------------------------------------------------¶  --|¬FUNCTION INTEGER_IMAGE¶  FUNCTION INTEGER_IMAGE ( V :INTEGER )¬RETURN STRING IS¶  BEGIN¶    IF V < 0 THEN¶      RETURN '-' & INTEGER_IMAGE(- V);¶    ELSIF¬V >= 10 THEN¶      RETURN INTEGER_IMAGE ( V / 10 ) &¬INTEGER_IMAGE ( V MOD 10 );¶    ELSE¶      RETURN "" & CHARACTER'VAL ( CHARACTER'POS (¬'0' ) + V¬);¶    END IF;¶  END INTEGER_IMAGE;¶  --|----------------------------------------------------------------------------------------------------------------¶  --|¬PROCEDURE¬SCAN_GRAMMAR¶  PROCEDURE SCAN_GRAMMAR IS¶    STATE_SEQ¬¬: SEQ_TYPE¬:= GR_STATE_SEQ;¶    TE
0010 R_GO_SUM¬¬: INTEGER¬¬:= 0;¶    NONTER_GO_SUM¬¬: INTEGER¬¬:= 0;¶    REDUCE_SUM¬¬: INTEGER¬¬:= 0;¶¶    --|----------------------------------------------------------------------------------------------------------------¶    --|¬PROCEDURE¬SCAN_STATE¶    PROCEDURE SCAN_STATE IS¶      ITEM_SEQ¬¬: SEQ_TYPE¬:= LIST (¬STATE );¶      ITEM¬¬: TREE;¶      SYL_SEQ¬¬: SEQ_TYPE;¶      SYL¬¬¬: TREE;¶      SYL_NBR¬¬: INTEGER;¶      RULE¬¬: TREE;¶      GOTO_STATE¬¬: TREE;¶¶      --|----------------------------------------------------------------------------------------------------------------¶      --|¬PROCEDURE¬CHECK_REDUCE¶      PROCEDURE CHECK_REDUCE ( ITEM :TREE ) IS¶      -- MARK SYMBOLS USED FOR REDUCE; CHECK FOR REDUCE-REDUCE CONFLICT¶        NBR_TERS¬¬: INTEGER¬¬:= 0;¶        FOLLOW_SEQ¬¬: SEQ_TYPE¬:= LIST( D( XD_FOLLOW, ITEM )¬);¶        TER¬¬: TREE;¶        TER_NBR¬¬: INTEGER;¶      BEGIN¶        WHILE NOT IS_EMPTY( FOLLOW_SEQ ) LOOP¶¬TER := HEAD( FOLLOW_SEQ ); FOLLOW_SEQ := TAIL( FOLLOW_SEQ );¶¬TER_NBR := DI( XD_TER_NBR, TER );¶¬IF SYLTBL( -TER_NBR¬).STATE_NBR /= STATE_NBR THEN¶¬  SYLTBL(¬-TER_NBR ).STATE_NBR := STATE_NBR;¶¬ELSIF SYLTBL( -TER_NBR ).REDUCE THEN¶¬  ERROR( D( LX_SRCPOS, D( XD_ALTERNATIVE, ITEM ) ),¶¬          "RED/RED CONF STATE " & INTEGER_IMAGE( STATE_NBR )¶¬¬¬& " - " &¬PRINT_NAME( D( XD_SYMREP, TER¬) ) );¶¬END IF;¶¬SYLTBL( -TER_NBR ).REDUCE := TRUE;¶¬NBR_TERS := NBR_TERS + 1;¶        END LOOP;¶        REDUCE_NBR_TERS( REDUCE_COUNT )¬:= NBR_TERS;¶        REDUCE_ITEM( REDUCE_COUNT ) := ITEM;¶      END¬CHECK_REDUCE;¶      --|----------------------------------------------------------------------------------------------------------------¶      --|¬¬FUNCTION REDUCE_ACTION¶      FUNCTION REDUCE_ACTION ( ITEM :TREE ) RETURN INTEGER IS¶        ALT¬¬: TREE¬:= D ( XD_ALTERNATIVE, ITEM );¶        ALT_NBR¬¬: INTEGER¬:= DI( XD_ALT_NBR, ALT );¶        ALT_SEM¬¬: INTEGER¬:= ALT_SEM_TBL( ALT_NBR );¶      -- 0 OR ACTION ENTRY¶        SEM_S¬¬: SEQ_TYPE;¶        SEM¬¬: TREE;¶  SEM_OP_POS¬¬: INTEGER;¶        SEM_OP_KIND¬: GRMR_OP;¶        CODE¬¬: INTEGER;¶          TXT¬¬: TREE;¶¶      --|----------------------------------------------------------------------------------------------------------------¶      --|¬PROCEDURE¬REDUCE_CODE¶        FUNCTION REDUCE_CODE ( ALT :TREE ) RETURN¬INTEGER IS¶¬NBR_POPS¬: INTEGER¬:= 0;¶¬SYL_LIST¬: SEQ_TYPE¬:= LIST (¬ALT );¶        BEGIN¶¬WHILE NOT¬IS_EMPTY(¬SYL_LIST ) LOOP¶¬  SYL_LIST := TAIL(¬SYL_LIST );¶¬  NBR_POPS := NBR_POPS + 1;¶¬END LOOP;¶¬RETURN - ( 10_000 +¬NBR_POPS * 1000 + DI ( XD_RULE_NBR, D (¬XD_RULEINFO, D ( XD_RULE, ALT¬) ) )¶¬          );¶      END REDUCE_CODE;¶¶      BEGIN¶      IF ALT_SEM /= 0 THEN¶¬RETURN ALT_SEM; -- ALREADY COMPUTED¶        END IF;¶¶        SEM_S := LIST( D( XD_SEMANTICS,¬ALT ) );¶        IF IS_EMPTY( SEM_S ) THEN¬¬¬¬¬¬¬¬¬ NO SEMANTICS, JUST USE REDUCE CODE¶¬ALT_SEM := REDUCE_CODE( ALT );¶        ELSE¬¬¬¬¬¬¬¬¬¬ -- SEMANTICS, INDIRECT INTO REST OF ALT¬TBL¶¬ALT_SEM := - (GRMR.AC_TBL_LAST + 1);¬¬¬¬¬¬¬¬ BRANCH¬TO WHERE SEMANTICS WILL START¶¬WHILE NOT¬IS_EMPTY(¬SE
0030 M_S) LOOP¶¬  POP( SEM_S, SEM );¶¬  GRMR.AC_TBL_LAST := GRMR.AC_TBL_LAST + 1;¶¬  SEM_OP_POS          := DI( XD_SEM_OP, SEM );¶¬  SEM_OP_KIND      := GRMR_OP'VAL( SEM_OP_POS );¶¬  CODE¬          := 1000 * SEM_OP_POS;¶¬  IF SEM_OP_KIND IN¬GRMR_OP_NODE THEN¶¬    CODE := CODE + DI( XD_KIND, SEM );¶¬  ELSIF SEM_OP_KIND¬IN GRMR_OP_QUOTE THEN¶¬    TXT  := D( XD_KIND, SEM );¶¬    CODE := CODE + INTEGER( TXT.PG );¶¬    GRMR.AC_TBL( GRMR.AC_TBL_LAST ) := AC_SHORT( CODE );¶¬    GRMR.AC_TBL_LAST := GRMR.AC_TBL_LAST + 1;¶¬    CODE := INTEGER( TXT.LN );¶¬  END IF;¶¬  GRMR.AC_TBL( GRMR.AC_TBL_LAST ) := AC_SHORT( CODE );¶¬END LOOP;¶¬GRMR.AC_TBL_LAST := GRMR.AC_TBL_LAST + 1;¶¬GRMR.AC_TBL( GRMR.AC_TBL_LAST¬) := AC_SHORT( REDUCE_CODE ( ALT ) );¶        END IF;¶¬    -- SAVE COMPUTED VALUE AND RETURN¶        ALT_SEM_TBL( ALT_NBR ) := ALT_SEM;¶        RETURN ALT_SEM;¶¶      END¬REDUCE_ACTION;¶      --|----------
----------------------------------------------------------------------------------------------------------------¶      --|¬PROCEDURE¬GEN_TER_INFO¶      PROCEDURE GEN_TER_INFO IS¶¬    -- FILL IN INFO FOR TERMINALS AND¬DONT CARE¬IN ACTION¬TABLE¶        ITEM_SEQ¬¬: SEQ_TYPE;¶        ITEM¬¬: TREE;¶        GOTO_STATE¬¬: TREE;¶  TEMP_INTEGER¬: INTEGER;¶        TEMP_TREE¬¬: TREE;¶        SYL_LIST¬¬: SEQ_TYPE;¶        SYL¬¬: TREE;¶        SYL_NBR¬¬: INTEGER;¶      BEGIN¶¬  WRITE TER GOTO ACTIONS¶        IF TER_GO_COUNT > 0 THEN¶¬ITEM_SEQ := LIST( STATE );¶¬WHILE NOT¬IS_EMPTY(¬ITEM_SEQ ) LOOP¶¬  ITEM :=¬HEAD( ITEM_SEQ );
0040  ITEM_SEQ :=¬TAIL( ITEM_SEQ );¶¬  GOTO_STATE := D( XD_GOTO, ITEM);¶¬  IF GOTO_STATE.TY /= DN_VOID¬THEN¶¬    SYL := HEAD( LIST( ITEM )¬);¶¬    IF SYL.TY = DN_TERMINAL THEN¶¬      SYL_NBR := DI¬( XD_TER_NBR, SYL );¶¬      IF SYLTBL( SYL_NBR ).STATE_NBR /= STATE_NBR THEN¶¬        SYLTBL( -SYL_NBR).STATE_NBR := 0;¶¬        GRMR.AC_SYM_LAST := GRMR.AC_SYM_LAST + 1;¶¬        GRMR.AC_SYM( GRMR.AC_SYM_LAST )¬:= AC_BYTE( SYL_NBR¬);¶¬        GRMR.AC_TBL( GRMR.AC_SYM_LAST )¬:= AC_SHORT( DI ( XD_STATE_NBR, GOTO_STATE ) );¶¬      END¬IF;¶¬  END IF;¶¬END LOOP;¶        END IF;¶¶¬      -- WRITE NON-DEFAULT REDUCE ACTIONS¶¬      -- FIRST MAKE¬#1 LONGEST REDUCE (IT WILL BE¬DONT CARE)¶        FOR I IN 2 .. REDUCE_COUNT LOOP¶¬IF REDUCE_NBR_TERS (I) > REDUCE_NBR_TERS (1) THEN¶¬  TEMP_INTEGER := REDUCE_NBR_TERS(1);¶¬  TEMP_TREE := REDUCE_ITEM(1);¶¬  REDUCE_NBR_TERS(1) := REDUCE_NBR_TERS(I);¶¬  REDUCE_ITEM(1) :=¬REDUCE_ITEM(I);¶¬  REDUCE_NBR_TERS(I) := TEMP_INTEGER;¶¬  REDUCE_ITEM(I) :=¬TEMP_TREE;¶¬END IF;¶¶¬¬    -- NOW COMPUTE REDUCE ACTION AND PUT OUT FOR EACH TER¶¬TEMP_INTEGER := REDUCE_ACTION( REDUCE_ITEM(I) );¶¬SYL_LIST := LIST ( D ( XD_FOLLOW, REDUCE_ITEM( I ) ) );¶¬WHILE NOT¬IS_EMPTY ( SYL_LIST¬) LOOP¶¬  GRMR.AC_SYM_LAST := GRMR.AC_SYM_LAST + 1;¶¬  GRMR.AC_SYM( GRMR.AC_SYM_LAST ) := AC_BYTE( DI ( XD_TER_NBR, HEAD (¬SYL_LIST ) ) );¶¬  GRMR.AC_TBL( GRMR.AC_SYM_LAST ) := AC_SHORT( TEMP_INTEGER¬);¶¬  SYL_LIST := TAIL ( SYL_LIST¬);¶¬END LOOP;¶¶        END LOOP;¶¶¬      -- NOW PUT OUT D

0050 ONT CARE ACTION¶        GRMR.AC_SYM_LAST := GRMR.AC_SYM_LAST + 1;¶        GRMR.AC_SYM( GRMR.AC_SYM_LAST )¬:= 0;¶        IF REDUCE_COUNT > 0 THEN¶¬GRMR.AC_TBL( GRMR.AC_SYM_LAST¬) := AC_SHORT( REDUCE_ACTION ( REDUCE_ITEM( 1 ) )¬);¶        ELSE¶¬GRMR.AC_TBL( GRMR.AC_SYM_LAST¬) := 0;¶        END IF;¶    END¬GEN_TER_INFO;¶¶    BEGIN¶      WHILE NOT IS_EMPTY( ITEM_SEQ) LOOP¶        POP( ITEM_SEQ, ITEM );¶        SYL_SEQ := LIST( ITEM¬);¶        IF IS_EMPTY( SYL_SEQ)¬THEN¶¬REDUCE_COUNT := REDUCE_COUNT + 1;¶¬CHECK_REDUCE( ITEM );¶        ELSE¶¬SYL := HEAD( SYL_SEQ );¶¬IF SYL.TY¬= DN_TERMINAL THEN¶¬  SYL_NBR¬:= - DI (¬XD_TER_NBR, SYL );¶¬  IF SYLTBL( SYL_NBR ).STATE_NBR /= STATE_NBR THEN¶¬    TER_GO_COUNT :=¬TER_GO_COUNT + 1;¶¬    SYLTBL( SYL_NBR¬).STATE_NBR := STATE_NBR;¶¬    SYLTBL( SYL_NBR¬).REDUCE := FALSE;¶¬  END IF;¶¬ELSE¶¬  RULE :=¬D( XD_RULE, SYL );¶¬  IF RULE.TY /= DN_VOID THEN¶¬    SYL_NBR := DI( XD_RULE_NBR, D( XD_RULEINFO, RULE ) );¶¬    IF SYLTBL( SYL_NBR ).STATE_NBR /= STATE_NBR THEN¶¬      NONTER_GO_COUNT := NONTER_GO_COUNT + 1;¶¬      SYLTBL( SYL_NBR ).STATE_NBR := STATE_NBR;¶¬      SYLTBL( SYL_NBR ).REDUCE := FALSE;¶¬    END IF;¶¬  END IF;¶¬END IF;¶        END IF;¶    END¬LOOP;¶¶¬      -- CHECK FOR SHIFT-REDUCE CONFLICTS¶      IF REDUCE_COUNT > 0 THEN¶        ITEM_SEQ :=¬LIST ( STATE );¶        WHILE NOT IS_EMPTY ( ITEM_SEQ )¬LOOP¶¬POP ( ITEM_SEQ, ITEM );¶¬SYL_SEQ := LIST ( ITEM );¶¬IF NOT IS_EMPTY ( SYL_SEQ ) THEN¶¬  SYL := HEAD ( SYL_SEQ );¶¬  IF SYL.TY = DN_TER
0060 MINAL THEN¶¬    SYL_NBR := - DI¬( XD_TER_NBR, SYL );¶¬    IF SYLTBL( SYL_NBR ).REDUCE THEN¶¬      ERROR ( D ( LX_SRCPOS, SYL ), "SHIFT/RED CONF STATE "¬& INTEGER_IMAGE ( STATE_NBR )¶¬¬¬& " - " &¬PRINT_NAME ( D ( XD_SYMREP, SYL ) ) );¶¬    END IF;¶¬  END IF;¶¬END IF;¶        END LOOP;¶      END¬IF;¶¶¬      -- WRITE NONTER ACTIONS¶      IF NONTER_GO_COUNT > 0 THEN¶        ITEM_SEQ :=¬LIST( STATE );¶        WHILE NOT IS_EMPTY( ITEM_SEQ ) LOOP¶¬POP( ITEM_SEQ, ITEM¬);¶¬GOTO_STATE := D ( XD_GOTO, ITEM );¶¬IF GOTO_STATE.TY /=¬DN_VOID THEN¶¬  SYL := HEAD( LIST¬( ITEM ) );¶¬  IF SYL.TY = DN_NONTERMINAL THEN¶¬    SYL_NBR := DI( XD_RULE_NBR, D( XD_RULEINFO, D( XD_RULE,¬SYL ) ) );¶¬    IF SYLTBL( SYL_NBR ).STATE_NBR = STATE_NBR THEN¶¬      SYLTBL( SYL_NBR ).STATE_NBR := 0;¶¬      GRMR.AC_SYM_LAST := GRMR.AC_SYM_LAST + 1;¶¬      GRMR.AC_SYM( GRMR.AC_SYM_LAST ) := AC_BYTE(¬SYL_NBR );¶¬      GRMR.AC_TBL( GRMR.AC_SYM_LAST ) := AC_SHORT( DI( XD_STATE_NBR, GOTO_STATE¬) );¶¬    END IF;¶¬  END IF;¶¬END IF;¶        END LOOP;¶      END¬IF;¶¶¬      -- WRITE STATE TABLE ENTRY¶      GRMR.ST_TBL_LAST := GRMR.ST_TBL_LAST + 1;¶¬      -- ASSUME NO SEMANTICS FOR NOW !!!!¶      IF TER_GO_COUNT = 0 AND¬THEN NONTER_GO_COUNT = 0 AND THEN REDUCE_COUNT = 1 THEN¶        GRMR.ST_TBL( GRMR.ST_TBL_LAST )¬:= REDUCE_ACTION( ITEM );¶      ELSE¶        GRMR.ST_TBL( GRMR.ST_TBL_LAST )¬:= GRMR.AC_SYM_LAST¬+ 1;¶        GEN_TER_INFO;¶      END¬IF;¶    END SCAN_STATE;¶¶  BEGIN¶    WHILE¬NOT IS_EMPTY ( STATE_SEQ ) LOOP¶
0070  POP¬( STATE_SEQ, STATE );¶        STATE_NBR := DI ( XD_STATE_NBR, STATE );¶        TER_GO_COUNT := 0;¶        NONTER_GO_COUNT := 0;¶        REDUCE_COUNT := 0;¶        SCAN_STATE;¶        INT_IO.PUT ( STATE_NBR );¶        INT_IO.PUT ( TER_GO_COUNT );¶        INT_IO.PUT ( NONTER_GO_COUNT );¶        INT_IO.PUT ( REDUCE_COUNT );¶        NEW_LINE;¶        NONTER_GO_SUM¬:= NONTER_GO_SUM + NONTER_GO_COUNT;¶        TER_GO_SUM :=¬TER_GO_SUM + TER_GO_COUNT;¶        REDUCE_SUM :=¬REDUCE_SUM + REDUCE_COUNT;¶    END LOOP;¶    PUT (¬"******" );¶    INT_IO.PUT ( TER_GO_SUM );¶    INT_IO.PUT ( NONTER_GO_SUM );¶    INT_IO.PUT ( REDUCE_SUM );¶    NEW_LINE;¶  END SCAN_GRAMMAR;¶  --|----------------------------------------------------------------------------------------------------------------¶  --|¬PROCEDURE¬WRITE_TABLES¶  PROCEDURE WRITE_TABLES IS¶    OFILE¬¬¬: FILE_TYPE;¶    STATE_IND¬¬: INTEGER¬:= 1;¶    RULE_LIST¬¬: SEQ_TYPE;¶    RULE¬¬¬: TREE;¶    AC_SUB¬¬: INTEGER;¶    TXT_LN¬¬: INTEGER;¶  BEGIN¶    CREATE( OFILE, OUT_FILE, "parse.tbl" );¶    PUT( "NBR OF STATES IS" );¶    INT_IO.PUT( GRMR.ST_TBL_LAST );¶    PUT( " - MAX");¶    INT_IO.PUT( GRMR.ST_TBL'LAST );¶    NEW_LINE;¶    PUT( "NBR OF ACTION SYMBOLS IS" );¶    INT_IO.PUT( GRMR.AC_SYM_LAST );¶    PUT( " - MAX" );¶    INT_IO.PUT( GRMR.AC_SYM'LAST );¶    NEW_LINE;¶    PUT( "LAST ACTION ENTRY IS" );¶    INT_IO.PUT( GRMR.AC_TBL_LAST );¶    PUT( " - MAX" );¶    INT_IO.PUT( GRMR.AC_TBL'LAST );¶    NEW_LINE;¶    FOR I¬IN 1 .. GRMR.AC_SYM_LAST LOOP¶      WHILE STATE_IND <= GRMR.ST_TBL_LAST AND THEN GRMR.ST_TBL( STATE_IND ) <= I LOOP¶        PUT( OFILE,¬'S' );¶        INT_IO.PUT(¬OFILE, STATE_IND, 4¬);¶        INT_IO.PUT(¬OFILE, GRMR.ST_TBL(¬STATE_IND¬) );¶        NEW_LINE( OFILE );¶        STATE_IND := STATE_IND + 1;¶      END¬LOOP;¶      PUT( OFILE, 'T' );¶      INT_IO.PUT( OFILE, I, 5¬);¶      INT_IO.PUT( OFILE, INTEGER( GRMR.AC_TBL( I ) ) );¶      INT_IO.PUT( OFILE, INTEGER( GRMR.AC_SYM( I ) ) );¶      NEW_LINE( OFILE );¶    END LOOP;¶    WHILE¬STATE_IND¬<= GRMR.ST_TBL_LAST¬LOOP¶      PUT¬( OFILE, 'S' );¶      INT_IO.PUT( OFILE, STATE_IND, 4 );¶      INT_IO.PUT( OFILE, GRMR.ST_TBL( STATE_IND )¬);¶      NEW_LINE( OFILE );¶      STATE_IND := STATE_IND + 1;¶    END LOOP;¶¶    PUT (¬"NUMBER OF ACTION ENTRIES IS"¬);¶
INT_IO.PUT( GRMR.AC_TBL_LAST );¶    NEW_LINE;¶¶    AC_SUB := GRMR.AC_SYM'LAST;¶    WHILE¬AC_SUB < GRMR.AC_TBL_LAST LOOP¶      AC_SUB := AC_SUB + 1;¶    PUT( OFILE, 'A' );¶      INT_IO.PUT( OFILE, AC_SUB, 5 );¶      DECLARE¶      DATA¬¬: INTEGER¬¬:= INTEGER( GRMR.AC_TBL( AC_SUB ) );¶        DATA_KIND¬¬: GRMR_OP;¶        TXT¬¬: TREE;¶      BEGIN¶        IF DATA < 1000 THEN¶¬INT_IO.PUT ( OFILE,¬DATA );¶        ELSIF (DATA¬/ 1000) >¬GRMR_OP'POS ( GRMR_OP'LAST ) THEN¶¬INT_IO.PUT ( OFILE,¬DATA );¶¬PUT( OFILE, "################");¶¬PUT_LINE(¬"##### ERROR IN TABLE");¶        ELSE¶¬DATA_KIND¬:= GRMR_OP'VAL( DATA / 1000 );¶¬IF DATA_KIND NOT IN¬GRMR_OP_QUOTE THEN¶¬  INT_IO.PUT( OFILE, DATA );¶¬ELSE¶¬  INT_IO.PUT( OFILE, GRMR_OP'POS( DATA_KIND ) * 1
0090 000 );¶¬  AC_SUB := AC_SUB + 1;¶¬  TXT_LN := INTEGER( GRMR.AC_TBL( AC_SUB ) );¶¬  IF TXT_LN IN 0..INTEGER( LINE_NBR'LAST ) THEN¶¬    TXT := (P, PG=>¬PAGE_IDX(¬DATA MOD 1000 ), TY=> DN_SYMBOL_REP, LN=> LINE_IDX( TXT_LN ));¶¬    PUT( OFILE, PRINT_NAME ( TXT ) );¶¬  ELSE¶¬    INT_IO.PUT ( OFILE, DATA MOD 1000 );¶¬    PUT( OFILE, ' '¬);¶¬    INT_IO.PUT ( OFILE, TXT_LN, 0 );¶¬    PUT( OFILE, "**********" );¶¬    PUT_LINE( "***** ERROR IN TABLE" );¶¬  END IF;¶¬END IF;¶        END IF;¶      END;¶      NEW_LINE ( OFILE );¶    END LOOP;¶¶    GRMR.NTER_LAST := 0;¶    RULE_LIST := LIST ( D ( XD_GRAMMAR,¬USER_ROOT));¶    WHILE¬NOT IS_EMPTY ( RULE_LIST ) LOOP¶      POP¬( RULE_LIST, RULE );¶      GRMR.NTER_LAST := GRMR.NTER_LAST + 1;¶      PUT¬( OFILE, 'N' );¶      INT_IO.PUT ( OFILE, GRMR.NTER_LAST, 4 );¶      PUT¬( OFILE, ' ' );¶      PUT_LINE ( OFILE, PRINT_NAME ( D ( XD_NAME,¬RULE ) ) );¶    END LOOP;¶    PUT (¬"NUMBER OF NONTERMINALS IS");¶    INT_IO.PUT ( GRMR.NTER_LAST );¶    NEW_LINE;¶    CLOSE¬( OFILE );¶  END WRITE_TABLES;¶¶BEGIN¶  OPEN_IDL_TREE_FILE( NOM_TEXTE & ".lar" );¶  USER_ROOT := D( XD_USER_ROOT, TREE_ROOT );¶  GR_STATE_SEQ := LIST( D( XD_STATELIST, USER_ROOT ) );¶¶  IF DI( XD_ERR_COUNT, TREE_ROOT ) > 0 THEN¶    INT_IO.PUT( DI(¬XD_ERR_COUNT, TREE_ROOT ), 1 );¶    PUT_LINE (  " ERRORS IN EARLY PHASES." );¶  END IF;¶¶  FOR I IN SYLTBL'RANGE LOOP¶    SYLTBL( I ).STATE_NBR := 0;¶  END LOOP;¶¶  GRMR.ST_TBL_LAST := 0;¶  GRMR.AC_SYM_LAST := 1;¶  GRMR.AC_TBL_LAST := GRMR.AC_SYM'L

0100 AST;¬¬¬-- I.E., NOTHING WITH GREATER¬INDEX YET¶  GRMR.AC_SYM( 1 ) := 0;¶  GRMR.AC_TBL( 1 ) := 0;¬¬¬-- ERROR AS FIRST ELT¶  GRMR.NTER_LAST :=¬0;¶¶  ALT_SEM_TBL := (OTHERS=> 0);¶¶  SCAN_GRAMMAR;¶  WRITE_TABLES;¶¶  CLOSE_IDL_TREE_FILE;¶--|----------------------------------------------------------------------------------------------------------------¶END CHECK_GRMR;¶
```